

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Artificial Intelligence and Data Science

ASSESSMENT TOOL 3 – CODING CHALLENGE

Course Code:	DSA03	Course Title:	Natural Language Processing
CO Assessed:	CO1 – Imitation (L1)	Assessment:	Assessment Tool 3 – Coding Challenge
Weightage:	30% of CIA		
CO5 Statement:	Analyse NLP models, regular expression patterns, finite state automata, morphological processing techniques and to interpret language processing. (BL-3)		
Student Name:	A. Mohith Krishna	Reg. No.:	192425199
Section / Batch:	2024	Date:	30/07/26

Code of Conduct

I _____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: _____

Instructions

- Develop a complete and modular Python program that meets all the functional requirements specified in the coding challenge using appropriate algorithms, data structures, and programming practices.
- Test the program using multiple valid and invalid test cases, and clearly display the input, processing, and output to verify the correctness of the implementation.
- Follow standard coding practices by using meaningful variable names, proper indentation, comments, exception handling (where applicable), and upload the source code, sample outputs, and README file to the designated GitHub repository.
- Duration: 40 minutes.

Section / Topic	Focus	Q Nos
Porter Stemmer	class design, methods, encapsulation, and rule-based stemming	1

Section 1: Object-Oriented Implementation of a Porter Stemmer

You have recently joined an **NLP startup** called **LexiMind Technologies**. The company is developing a **search engine** for a digital library containing millions of research papers. Users expect the search engine to return relevant results even when different grammatical forms of a word are used.

For example, a search for **connect** should also retrieve documents containing **connected**, **connecting**, **connection**, and **connections**. To achieve this, the development team has decided to implement the **Porter Stemming Algorithm** from scratch.

Since the algorithm consists of multiple sequential rules (Step 1a, Step 1b, Step 1c, Step 2, Step 3, Step 4, Step 5a, and Step 5b), the project manager has divided the work among team members. Each developer is responsible for implementing and testing one specific rule. At the end of the project, all modules must be integrated into a single, fully functional stemmer.

Assessment Task

Phase 1: Individual Task (Assigned Rule)

Each student will be assigned **one Porter Stemmer rule**.

Your responsibilities are to:

- Study and understand the assigned rule.
- Explain the linguistic purpose of the rule.
- Describe the conditions under which the rule should be applied.
- Implement the rule in Python.
- Prepare at least **10 test words** demonstrating:
 - Words that satisfy the rule.
 - Words that should remain unchanged.
- Document the input, expected output, and actual output.
- Explain any limitations or special cases encountered.

Phase 2: Team Integration

As a team, integrate all individual rule implementations into a complete Porter Stemmer.

The team must:

- Arrange the rules in the correct execution order.
- Resolve conflicts between rule implementations.
- Ensure that the output of one rule becomes the input to the next rule.
- Test the complete algorithm using at least **50 different English words**.
- Compare the team's results with the NLTK Porter Stemmer.
- Identify and explain any differences.

Phase 3: Reflection

Prepare a report addressing the following questions:

- Why is the order of Porter Stemmer rules important?
- What errors would occur if your assigned rule were skipped?
- How did integrating multiple rules improve stemming accuracy?
- Which rule was the most challenging to implement and why?
- What improvements would you suggest for the Porter Stemming algorithm?

Phase 1: Individual Task (Assigned Rule):

1. Step 1a – Plural Reduction

Purpose: Removes common plural endings to obtain a shorter stem.

Rules:

- sses → ss
- ies → i
- ss → ss
- s →

Examples:

Input	Expected Output
caresses	caress
ponies	poni
ties	ti
cats	cat
books	book
dogs	dog
boxes	box
kisses	kiss
class	class
cars	car

Condition: The rule is applied based on the word's ending. Words ending in **ss** remain unchanged.

2. Step 1b – ed and ing Removal

Purpose: Removes common verb endings such as **ed** and **ing**.

Examples:

Input	Output
agreed	agre
plastered	plaster
motoring	motor

troubled	troubl
sized	size
hopping	hop
tanned	tan
falling	fall
failing	fail
cleaned	clean

Condition: The remaining stem must contain a vowel. Additional changes may be made for double consonants or short words.

3. Step 1c – Final y Conversion

Purpose: Changes final y to i when the stem contains a vowel.

Input	Output
happy	happi
cry	cri
reply	repli
study	studi
carry	carri
fly	fli
try	tri
sky	sky
boy	boy
enjoy	enjoy

Special case: sky remains unchanged because its stem does not contain a vowel.

Step 2 – Derivational Suffix Reduction

Purpose: Removes longer suffixes such as ational, izer, fulness, and ization.

Examples:

Input	Output
relational	relat
conditional	condit
rational	ration
digitizer	digit
formalization	formal
hopefulness	hopeful
callousness	callous
feudalism	feudal
operator	operate
sensitiviti	sensitive

Condition: The stem must satisfy $m > 0$.

Step 3 – Further Suffix Reduction

Purpose: Removes additional suffixes to obtain a shorter stem.

Examples:

Input	Output
triplicate	triplic
formative	form

<u>formalize</u>	<u>formal</u>
<u>electrical</u>	<u>electric</u>
<u>hopeful</u>	<u>hope</u>
<u>goodness</u>	<u>good</u>
<u>kindness</u>	<u>kind</u>
<u>logical</u>	<u>logic</u>
<u>careful</u>	<u>care</u>
<u>useful</u>	<u>use</u>

Condition: Applied when the stem has $m > 0$.

Step 4 – Removing Derivational Suffixes

Purpose: Removes suffixes when the stem is sufficiently complex.

Examples:

<u>Input</u>	<u>Output</u>
<u>revival</u>	<u>reviv</u>
<u>allowance</u>	<u>allow</u>
<u>inference</u>	<u>infer</u>
<u>airliner</u>	<u>airlin</u>
<u>adjustable</u>	<u>adjust</u>
<u>defensible</u>	<u>defens</u>
<u>replacement</u>	<u>replac</u>

<u>adjustment</u>	<u>adjust</u>
-------------------	---------------

<u>adoption</u>	<u>adopt</u>
-----------------	--------------

<u>effective</u>	<u>effect</u>
------------------	---------------

Condition: The stem must have $m > 1$.

Step 5a – Removing Final e

Purpose: Removes an unnecessary final **e**.

<u>Input</u>	<u>Output</u>
--------------	---------------

<u>probate</u>	<u>probat</u>
----------------	---------------

<u>activate</u>	<u>activ</u>
-----------------	--------------

<u>effective</u>	<u>effect</u>
------------------	---------------

<u>operate</u>	<u>oper</u>
----------------	-------------

<u>create</u>	<u>creat</u>
---------------	--------------

<u>communicate</u>	<u>communic</u>
--------------------	-----------------

<u>rate</u>	<u>rate</u>
-------------	-------------

<u>cease</u>	<u>ceas</u>
--------------	-------------

<u>formalize</u>	<u>formal</u>
------------------	---------------

<u>motivate</u>	<u>motiv</u>
-----------------	--------------

Condition: Removal depends on the measure **m** and the CVC condition.

Step 5b – Double **l** Reduction

Purpose: Changes final **ll** to **l** when the stem is sufficiently complex.

<u>Input</u>	<u>Output</u>
--------------	---------------

<u>controll</u>	<u>control</u>
-----------------	----------------

<u>roll</u>	<u>roll</u>
-------------	-------------

<u>tall</u>	<u>tall</u>
-------------	-------------

<u>fall</u>	<u>fall</u>
-------------	-------------

<u>recall</u>	<u>recall</u>
---------------	---------------

<u>install</u>	<u>install</u>
----------------	----------------

<u>overall</u>	<u>overall</u>
----------------	----------------

<u>parallel</u>	<u>parallel</u>
-----------------	-----------------

successful success

patrol patrol

Condition: Applied when $m > 1$.

Phase 2 – Team Integration

Correct Rule Order

Step 1a

↓

Step 1b

↓

Step 1c

↓

Step 2

↓

Step 3

↓

Step 4

↓

Step 5a

↓

Step 5b

Each rule takes the output of the previous rule as its input.

Python Implementation

```
def porter_stem(word):  
    word = word.lower()  
    if word.endswith("sses"):  
        word = word[:-2]  
    elif word.endswith("ies"):  
        word = word[:-2]  
    elif word.endswith("ss"):  
        pass  
    elif word.endswith("s"):  
        word = word[:-1]  
    if word.endswith("eed"):  
        word = word[:-1]  
    elif word.endswith("ed") or word.endswith("ing"):  
        suffix = "ed" if word.endswith("ed") else "ing"  
        stem = word[:-len(suffix)]  
        if any(c in "aeiou" for c in stem):  
            word = stem  
    if word.endswith("y"):  
        word = word[:-1] + "i"  
    return word
```

Note: This short code demonstrates the basic rule-processing structure. For the complete project, all eight rules should be implemented.

Phase 3 – Reflection

1. Why is rule order important?

The rules are sequential. The output of one rule becomes the input to the next rule. Changing the order can produce incorrect stems.

2. What happens if a rule is skipped?

Some words will not be reduced correctly. For example, skipping Step 1a leaves plural words such as **cats** instead of producing **cat**.

3. How does integration improve accuracy?

Combining all rules allows the stemmer to handle plural, tense, and derivational forms together, giving better search results.

4. Which rule is challenging?

Step 4 is challenging because it contains many suffixes and uses the **m > 1** condition. Step 1b is also difficult because it contains several additional conditions.

5. Suggested improvements

- Add exception handling for irregular words.
 - Use lemmatization for better linguistic accuracy.
 - Support multiple languages.
 - Combine stemming with modern NLP techniques.
-

Rubrics for Calculation

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding	20%	Clearly understands problem constraints, edge cases, and competitive requirements	Understands problem with minor gaps in constraints	Partial understanding; misses key constraints	Misinterprets problem or constraints
Algorithm	25%	Selects optimal algorithm with best time and space complexity for competitive constraints	Uses efficient algorithm with minor scope for improvement	Uses basic/inefficient approach	No clear algorithmic approach
Code Implementation	20%	Writes correct, efficient, and clean code that passes all constraints	Code works with minor errors or inefficiencies	Partially correct code; fails for some cases	Code does not execute or gives incorrect results
Competitive Performance	20%	Solves problem within time limits and handles large inputs effectively	Solves within acceptable time with minor delays	Struggles with time limits or larger inputs	Unable to solve within time constraints
Test Case Handling	15%	Handles all test cases including edge and hidden cases	Handles most test cases	Misses important edge cases	Fails on multiple test cases

Q. No. / Criteria	Problem Understanding	Algorithm	Code Implementation	Competitive Performance	Test Case Handling	Sub. Total	Total %
1							
2							
3							

Faculty In-charge	Course Coordinator	Program Director
Dr.T.Kumaragurubaran		
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____

